Практика | Задание №24 ЕГЭ 2024

Уровень: "Важный"

Задание №1

Текстовый файл состоит из символов R, E, N, A, I, O.
Определите максимальное количество идущих подряд символов в прилагаемом файле, среди которых пара символов RO (в указанном порядке) встречается ровно 358 раз.

Прикрепи файл с программой для проверки задания куратором и краткий ответ!

Верные варианты

14755

Решение

Сообщить об ошибке

Ответ: 14755
1. Откроем файл, считаем строку, а также прибавим в начало и в конец строки подстроку 'RO'.
```
f = open("24.1.txt")
s = "RO" + f.readline() + "RO"
```
2. Создадим список ind_RO, в который будем добавлять все индексы подстрок 'RO'.
```
ind_RO = []
```
3. Запустим цикл перебора строки, в котором будем добавлять все индексы подстрок 'RO'.
```
for i in range(len(s)):
  if s[i:i + 2] == "RO":
    ind_RO.append(i)
```
INSANEK.RU
4. Создадим список answer для хранения количеств символов всех подстрок, которые содержат 358 пар символов 'RO'.
```
answer = []
```
5. Запустим цикл перебора всех индексов пар символов 'RO', внутри которого будем добавлять в список answer разность ind_RO[i + 359] - ind_RO[i], чтобы найти количества символов всех подходящих по условию подстрок.
```
for i in range(len(ind_RO) - 359):
  answer.append((ind_RO[i + 359] - ind_RO[i]))
```
6. Обработаем крайние случаи, в рамках которых код может включать несуществующие символы. Может быть две ситуации, которые следует обработать: у нас ровно один максимум в списке и он находится в начале или в конце строки, а также у нас есть ровно 2 максимума в строке и оба находятся по краям. В таких случаях нужно вычесть единицу, чтобы не учесть
```
if ((answer.count(max(answer)) == 1 and (answer[0] == max(answer) or answer[-1] == max(answer)))
    or (answer.count(max(answer)) == 2 and answer[0] == max(answer) and answer[-1] == max(answer))):
  print(max(answer) - 1)
else:
  print(max(answer))
```
Код:
```
f = open("24.1.txt")
s = "RO" + f.readline() + "RO"
ind_RO = []

for i in range(len(s)):
  if s[i:i + 2] == "RO":
    ind_RO.append(i)

answer = []
for i in range(len(ind_RO) - 359):
  answer.append((ind_RO[i + 359] - ind_RO[i]))

if ((answer.count(max(answer)) == 1 and (answer[0] == max(answer) or answer[-1] == max(answer)))
    or (answer.count(max(answer)) == 2 and answer[0] == max(answer) and answer[-1] == max(answer))):
  print(max(answer) - 1)
else:
  print(max(answer))
```

Задание №2

Текстовый файл 24.1.txt состоит из символов, обозначающих знаки «+», «*» и цифры 0, 1, 2, 3.Определите в прилагаемом файле максимальное количество идущих подряд символов, которые образуют математически правильную последовательность, в которую входят знаки «+» или «*» и натуральные числа без незначащих нулей.

Верные варианты

60

Решение

Сообщить об ошибке

Ответ: 60

Решение через регулярные выражения:

1. Импортируем библиотеку re для работы с регулярными выражениями.

import re

2. Откроем файл и считаем строку из файла в переменную s.

f = open("24.1.txt")

s = f.readline()

3. Создадим список is_valid, в который с помощью функции findall() добавим все подстроки, подходящие по условию.

is_valid = re.findall(r'(?:[123][0123]*)(?:[+*](?:[123][0123]*))*', s)

4. Выведем наибольшую по длине подстроку.

print(len(max(is_valid, key=len)))

Код:

```
import re
f = open("24.1.txt")
s = f.readline()
is_valid = re.findall(r'(?:[123][0123]*)(?:[+*](?:[123][0123]*))*', s)
print(len(max(is_valid, key=len)))
```

Решение перебором:

1. Откроем файл, считаем строку и заменим все арифметические знаки на знак умножения, затем разделим строку по этому символу.

f = open("24.1.txt")

s = f.readline()

s = s.replace("+", "*")

a = s.split("*")

INSANEK.RU

2. Создадим переменные arifm и max_arifm для хранения длины текущей и максимальной подстрок соответственно.

arifm = ""

max_arifm = ""

3. Запустим цикл для перебора всех подстрок. Внутри цикла сделаем проверку, что если длина текущей подстроки не равна нулю и первый символ не равен нулю, то прибавим к переменной текущую подстроку и знак умножения.

```
for x in a:
  if len(x) != 0 and x[0] != "0":
    arifm += x + "*"
```

4. Иначе если длина текущей подстроки больше нуля, но первый символ является нулем, а числовое значение подстроки не равно нулю, то приравняем к нашей строке подстроку без незначащих нулей и со знаком умножения.

```
  elif len(x) > 0 and x[0] == "0" and int(x) != 0:
    arifm = str(int(x)) + "*"
```

5. Иначе приравняем строку к пустой и обновим длину максимальной строки.

```
  else:
    arifm = ""
  max_arifm = max(arifm, max_arifm, key=len)
```

6. Выведем длину наибольшей подстроки и вычтем единицу.

print(len(max_arifm) - 1)

Код:

```
f = open("24.1.txt")
s = f.readline()
s = s.replace("+", "*")
a = s.split("*")
arifm = ""
max_arifm = ""
for x in a:
  if len(x) != 0 and x[0] != "0":
    arifm += x + "*"
  elif len(x) > 0 and x[0] == "0" and int(x) != 0:
    arifm = str(int(x)) + "*"
  else:
    arifm = ""
  max_arifm = max(arifm, max_arifm, key=len)
print(len(max_arifm) - 1)
```

Задание №3

Текстовый файл состоит из символов, обозначающих знаки «+», «*» и цифры 0, 2, 3, 4.Определите в прилагаемом файле максимальное количество идущих подряд символов, которые образуют математически правильную последовательность, в которую входят знаки «+» или «*» и натуральные числа без незначащих нулей.

Прикрепите не только краткий ответ, но и решение задачи в виде кода, чтобы куратор смог проверить его!

Решение

Сообщить об ошибке

Ответ: 62

Решение через регулярные выражения:

1. Импортируем библиотеку re для работы с регулярными выражениями.

import re

2. Откроем файл и считаем строку из файла в переменную s.

f = open("24.2.txt")
s = f.readline()

3. Создадим список is_valid, в который с помощью функции findall() добавим все подстроки, подходящие по условию.

is_valid = re.findall(r'(?:[234][0234]*)(?:[+*](?:[234][0234]*))*', s)

4. Выведем наибольшую по длине подстроку.

print(len(max(is_valid, key=len)))

Код:

```
import re
f = open("24.2.txt")
s = f.readline()
is_valid = re.findall(r'(?:[234][0234]*)(?:[+*](?:[234][0234]*))*', s)
print(len(max(is_valid, key=len)))
```

Решение перебором:

1. Откроем файл, считаем строку и заменим все арифметические знаки на знак умножения, затем разделим строку по этому символу.

f = open("24.2.txt")
s = f.readline()
s = s.replace("+", "*")
a = s.split("*")

INSANEK.RU

2. Создадим переменные arifm и max_arifm для хранения длины текущей и максимальной подстрок соответственно.

arifm = ""
max_arifm = ""

3. Запустим цикл для перебора всех подстрок. Внутри цикла сделаем проверку, что если длина текущей подстроки не равна нулю и первый символ не равен нулю, то прибавим к переменной текущую подстроку и знак умножения.

```
for x in a:
  if len(x) != 0 and x[0] != "0":
    arifm += x + "*"
```

4. Иначе если длина текущей подстроки больше нуля, но первый символ является нулем, а числовое значение подстроки не равно нулю, то приравняем к нашей строке подстроку без незначащих нулей и со знаком умножения.

```
  elif len(x) > 0 and x[0] == "0" and int(x) != 0:
    arifm = str(int(x)) + "*"
```

5. Иначе приравняем строку к пустой и обновим длину максимальной строки.

```
  else:
    arifm = ""
  max_arifm = max(arifm, max_arifm, key=len)
```

6. Выведем длину наибольшей подстроки и вычтем единицу.

print(len(max_arifm) - 1)

Код:

```
f = open("24.2.txt")
s = f.readline()
s = s.replace("+", "*")
a = s.split("*")
arifm = ""
max_arifm = ""
for x in a:
  if len(x) != 0 and x[0] != "0":
    arifm += x + "*"
  elif len(x) > 0 and x[0] == "0" and int(x) != 0:
    arifm = str(int(x)) + "*"
  else:
    arifm = ""
  max_arifm = max(arifm, max_arifm, key=len)
print(len(max_arifm) - 1)
```

Задание №4

Текстовый файл состоит из символов, обозначающих знаки «–», «*» и цифры 0, 3, 4, 5.
Определите в прилагаемом файле максимальное значение математического выражения, составленного из подряд идущих символов и которое включает знаки «–» или «*» и натуральные числа без незначащих нулей.
В ответе укажите число — значение найденного выражения.
Прикрепите не только краткий ответ, но и решение задачи в виде кода, чтобы куратор смог проверить его

Решение

Сообщить об ошибке

Ответ: 438046623591226771786623583200
Решение через регулярные выражения:
1. Импортируем библиотеку re для работы с регулярными выражениями:
from re import findall
2. Откроем файл и считаем строку из файла в переменную s:
f = open("24.txt")
s = f.readline()
3. Создадим список is_valid, в который с помощью функции findall() добавим все подстроки, подходящие по условию.
is_valid = findall("[345][0345]*(?:[*][345][0345]*)*", s)
4. Выведем наибольшее по значению выражение.
print(eval(max(is_valid, key=eval)))

Код:
```
from re import findall
f = open("24.txt")
s = f.readline()
is_valid = findall("[345][0345]*(?:[*][345][0345]*)*", s)
print(eval(max(is_valid, key=eval)))
```

INSANEK.RU

Задание №5

Текстовый файл 24hm.txt состоит из символов, обозначающих знаки «–», «*» и цифры 0, 7, 8, 9.
Определите в прилагаемом файле максимальное значение математического выражения, составленного из подряд идущих символов и которое включает знаки «–» или «*» и натуральные числа без незначащих нулей.
В ответе укажите число — значение найденного выражения.
Прикрепите не только краткий ответ, но и решение задачи в виде кода, чтобы куратор смог проверить его!

Решение

Сообщить об ошибке

Ответ: 597290135617060993890009836361748764649881978708277 8
Решение через регулярные выражения:
1. Импортируем библиотеку re для работы с регулярными выражениями.
import re
2. Откроем файл и считаем строку из файла в переменную s.
f = open("24.3.txt")
s = f.readline()
3. Создадим список is_valid, в который с помощью функции findall() добавим все подстроки, подходящие по условию.
is_valid = re.findall("[789][0789]*(?:[*][789][0789]*)*", s)
4. Выведем наибольшее по значению выражение.
Обратим внимание, что если включить минус выражение станет меньше!
print(eval(max(is_valid, key=eval)))
Код:
```
f = open("24hm.txt")
s = f.readline()
```

```
res = re.findall("[789][0789]*(?:[-*][789][0789]*)*", s)
print(max(res, key=eval))
print(eval(max(res, key=eval)))
```

Уровень: "Сложный"

Задание №6

Текстовый файл состоит из десятичных цифр, знаков «+» и «*» (сложения и умножения). Определите максимальное количество символов в непрерывной последовательности, являющейся корректным арифметическим выражением с целыми неотрицательными числами (без знака), значение которого равно нулю. В этом выражении никакие два знака арифметических операций не стоят рядом, порядок действий определяется по правилам математики. В записи чисел отсутствуют незначащие (ведущие) нули.
В ответе укажите количество символов.

Верные варианты

159

Решение

Сообщить об ошибке

Ответ: 159
1. Откроем файл и считаем строку из файла.
f = open('24.4.txt')
s = f.readline()
2. Во всех возможных парах символов арифметических знаков добавим пробел между знаками, затем разделим строку по пробелу.
s = s.replace("++", "+ +").replace("++", "+ +")
s = s.replace("", "* *").replace("**", "* *")**
s = s.replace("+*", "+ *").replace("+*", "+ *")
s = s.replace("*+", "* +").replace("*+", "* +")
a = s.split()
3. Создадим переменную max_buf, которая будет хранить в себе наибольшую длину необходимой по условию подстроки.
max_buf = ""
4. Запустим цикл для перебора всех подстрок. Создадим копию текущей подстроки.
for x in a:
x2 = x[:]
5. Если первый или последний символы подстроки является арифметическим знаком, то в копии подстроки уберем эти символы, затем разделим подстроку по символу плюса.
if x[0] in "+*":
x2 = x2[1:]
if x[-1] in "+*":
x2 = x2[:-1]
x2 = x2.split("+")
6. Создадим строку buf, которая будет хранить в себе длину подходящей по условию подстроки.
buf = ""
7. Запустим вложенный цикл для перебора подсумм в копии подстроки. Если длина подстроки будет больше нуля, а числовое значение выражения будет равно нулю, то прибавим текущую подстроку вместе с символом плюса.
for pod_summi in x2:
if len(pod_summi) > 0 and eval(pod_summi) == 0:
buf += pod_summi + "+"
8. Иначе если длина текущей подстроки больше нуля, а само выражение умножается на 0, то приравняем buf к строке "0*" + pod_summi.split("0*")[-1], иначе приравняем buf к пустой строке.
elif len(pod_summi) > 0:
if "0*" in pod_summi:
buf = "0*" + pod_summi.split("0*")[-1]
else:
buf = ""
9. Иначе приравняем buf к пустой строке, а также обновим длину максимума.
else:
buf = ""
max_buf = max(buf, max_buf, key=len)
10. Выведем длину максимальной строки и вычтем единицу.
print(len(max_buf) - 1)

INSANEK.RU

Код:

```
f = open('24.4.txt')
s = f.readline()
s = s.replace("++", "+ +").replace("++", "+ +")
s = s.replace("**", "* *").replace("**", "* *")
s = s.replace("+*", "+ *").replace("+*", "+ *")
s = s.replace("*+", "* +").replace("*+", "* +")
a = s.split()
max_buf = ""
for x in a:
    x2 = x[:]
    if x[0] in "+*":
        x2 = x2[1:]
    if x[-1] in "+*":
        x2 = x2[:-1]
    x2 = x2.split("+")
    buf = ""
    for pod_summi in x2:
        if len(pod_summi) > 0 and eval(pod_summi) == 0:
            buf += pod_summi + "+"
        elif len(pod_summi) > 0:
            if "0*" in pod_summi:
                buf = "0*" + pod_summi.split("0*")[-1]
            else:
                buf = ""
        else:
            buf = ""
        max_buf = max(buf, max_buf, key=len)
print(len(max_buf) - 1)
```

INSANEK.RU